

OOP VỚI JAVA

ThS. Nguyễn Minh Đế

denm81@gmail.com

Vấn đề

- **Yêu cầu:** chương trình quản lý Sinh viên:

STT	Mã SV	Họ đệm	Tên	Lớp học	Toán	Lý
1	1019	Phan Hoàng Quốc	Bảo	10DHTH9	9.0	5.0
2	1010	Lương Thành	Đạt	10DHTH6	v	v
3	1039	Trần Thị	Hằng	10DHTH9	10	5.0
4	1002	Bùi Minh	Hoàng	10DHTH6	10	4.5
5	1032	Đinh Thành	Long	10DHTH9	10	5.0
6	0690	Nguyễn Hiếu	Nghĩa	10DHTH6	8.0	5.5
7	1901	Trần Hiếu	Nghĩa	10DHTH9	10.0	5.0
8	0699	Tạ Hoàng Bảo	Nguyên	10DHTH6	5.0	2.0
9	0706	Võ Nguyễn Thành	Nhân	10DHTH6	10	4.5
10	1015	Đỗ Huy	Phương	10DHTH6	6.0	5.0

denm81@gmail.com

Vấn đề

- **Yêu cầu:** chức năng
  a) Thêm mới, Xuất ra SV
  b) Xóa SV theo MSV
  c) Tìm theo MSV
  d) → lập trình với C ?

```
struct  SV{
int STT; int MSV;
char HoDem[40]; char Ten[40];
char LopHoc[20];
float Toan, Ly;
};
```

denm81@gmail.com

Vấn đề

- **Yêu cầu:** Thêm mới SV

```
void NhapSV(SV SV1[], int n) { //nhập vào mảng
  for (int i = 0; i < n; i++) {
        SV1[i].STT = i;printf("\nSTT: %d", SV1[i].STT);
        printf("\n MSV: "); scanf("%d", &SV1[i].MSV);
        printf("\n Ho Dem: "); gets_s(SV1[i].HoDem);
        printf("\n Ten: ");gets_s(SV1[i].Ten);
        printf("\n Lop Hoc: "); gets_s(SV1[i].LopHoc);
        printf("\n Toan: "); scanf("%d", &SV1[i].Toan);
        printf("\n Ly: "); scanf("%d", &SV1[i].Ly);
}
}
```


denm81@gmail.com

Vấn đề

- **Yêu cầu:** xuất SV

```
void XuatSV(SV SV1[], int n) { // xuất ra
    for (int i = 0; i < n; i++) {
        // lệnh
    }
}
```

- **Tương tự:** mỗi một yêu cầu chức năng là 1 hàm riêng biệt
- → phân công cho các lập trình viên LTV

denm81@gmail.com

Vấn đề

- **Yêu cầu:** từ người dùng (nhà trường)
  - SV có thêm (Địa chỉ, Môn hóa, ..)
  - Thêm chức năng: tìm theo Tên ?
  - → cách xử lý yêu cầu và lập trình ?
  - Xây dựng struct mới ? Thêm vào struct cũ
  - Thêm hàm mới ? Chỉnh sửa hàm cũ ?

```
struct  SV{
int STT; int MSV;
char HoDem[40]; char Ten[40];
char LopHoc[20]; char DiaChi[20];
float Toan, Ly, Hoa;
};
```


denm81@gmail.com

Vấn đề

- **Đặc tính:** Lập trình cấu trúc
  - CT chia nhỏ thành các chương trình con (hàm, thủ tục) thực hiện các công việc rời rạc
  - Các hàm giữ càng độc lập với nhau
  - Mỗi hàm có dữ liệu và logic riêng.
  - Thông tin giữa các hàm thông qua các tham số
  - Các hàm có thể có các biến cục bộ (bên ngoài phạm vi của hàm lại có thể truy xuất được chúng)
  - các hàm được xem là các chương trình con được đặt chung với nhau → một ứng dụng lớn.

denm81@gmail.com

Vấn đề

- **Điểm yếu:** Lập trình cấu trúc
  - khi kích thước của chương trình càng tăng.
  - → Các kiểu dữ liệu xử lý trong nhiều hàm khác nhau bên trong chương trình có cấu trúc.
  - có sự thay đổi trong các dữ liệu này → thực hiện cả các thay đổi ở mọi nơi.
  - → tốn thời gian và kém hiệu quả đối với các chương trình có hàng ngàn dòng lệnh và hàng trăm hàm trở lên.

denm81@gmail.com

Vấn đề

- **Điểm yếu:** Lập trình cấu trúc

  – khi có nhiều lập trình viên làm việc theo nhóm

  – → được phân công viết một tập hợp các hàm và các kiểu dữ liệu.

  – nhiều lập trình viên khác nhau quản lý các hàm riêng

  – → có liên quan đến các kiểu dữ liệu dùng chung

  – → các thay đổi mà LTV tạo ra trên một phần tử dữ liệu → ảnh hưởng đến tất cả các người còn lại

  – → sai sót trong việc trao đổi thông tin giữa các thành viên trong nhóm có thể dẫn tới hậu quả là mất rất nhiều thời gian để sửa chữa chương trình.

denm81@gmail.com

– suy nghĩ "*làm thế nào để giải quyết công việc*" ?

– tư duy "*với một số đối tượng đã có, phải làm gì để giải quyết được công việc đặt ra*"

– → *làm cái gì với một số đối tượng đã có đó*

– một chiếc xe: các bánh xe, di chuyển được, đổi hướng (quẹo tay lái, …).

– một cái cây là một loại thực vật có thân gỗ và lá.

– Một chiếc xe không phải là một cái cây, mà cái cây không phải là một chiếc xe

– làm được với một chiếc xe thì không thể làm được với một cái cây.

- Lập trình hướng đối tượng:

  – tổ chức dữ liệu trong chương trình như các nhà sinh học tổ chức các sinh vật (loại thực vật, ..) khác nhau.

  – Xe hơi, cây cối, SV, các quyển sách, … đều được gọi là các **lớp** (**Class**).

  – Một lớp là một bản mẫu mô tả các thông tin cấu trúc dữ liệu, lẫn các thao tác hợp lệ của các phần tử dữ liệu.

Lớp (Class)
Dữ liệu: …
Thao tác hợp lệ: …

denm81@gmail.com

LỚP (CLASS)

- Lớp được xem như một khuôn mẫu (template) của đối tượng (Object).
- Trong lớp bao gồm các thuộc tính của đối tượng (properties) và các phương thức (methods) tác động lên các thuộc tính.
- Đối tượng được xây dựng từ lớp nên được gọi là thể hiện của lớp (class instance).

denm81@gmail.com

- Lập trình hướng đối tượng:
  - Khi một phần tử dữ liệu được khai báo là phần tử của một lớp thì nó được gọi là một **đối tượng (Object)**.
  - Các hàm được định nghĩa hợp lệ trong một lớp được gọi là các **phương thức (Method)**: các hàm xử lý dữ liệu của các đối tượng của lớp đó.
  - Các dữ liệu (thông tin) là thuộc tính
  - có thực bên trong bộ nhớ, thực chất đó là một đối tượng (một đối tượng được cấp phát vùng nhớ)

denm81@gmail.com

So sánh

- Hướng đối tượng >< lập trình cấu trúc :
  - Chương trình = Cấu trúc dữ liệu + Thuật giải
  - Đối tượng = Phương thức + Dữ liệu

```
struct  SV{
int STT; int MSV;
char HoDem[40]; float
Toan, Ly; };
```

Lớp (Class)	**SV**
Dữ liệu, …	STT, MSV, …
Thao tác hợp lệ, …	Xuất, nhập, xóa, …

```
void XuatSV(SV SV1[],
int n) { // xuất ra
        // lệnh
}
```

Đối tượng
01, **099921,** …
Xuất, nhập, xóa, …

denm81@gmail.com
Khoa Công Nghệ Thông Tin

Trừu Tượng Dữ Liệu

Các tính chất của một đối tượng Người

Tên
Địa chỉ
Tuổi
Chiều cao
Màu tóc

Các tính chất của một đối tượng Khách hàng

Tên
Địa chỉ

Trừu Tượng Dữ Liệu (tiếp theo)

Các thuộc tính	Các hành động
Tên của khách hàng	Nhập tên của khách hàng
Địa chỉ của khách hàng	Nhập địa chỉ của khách hàng
Đời xe hơi đã mua	Nhập đời xe hơi mua được
Người bán xe hơi	Nhập tên người bán xe hơi
	Lập hoá đơn

Lớp

- Lớp là một nhóm các đối tượng có chung những tính chất và hành động

Lớp Khách hàng
Tên khách hàng
Địa chỉ khách hàng
Đời xe hơi đã mua
Tên người bán xe hơi
Nhập tên()
Nhập địa chỉ()
Nhập đời của xe hơi mua được()
Nhập tên của người bán xe hơi()
Lập hóa đơn()

Đối Tượng

- Đối tượng là một thể hiện của lớp
  - Toàn
  - Anh
  - Tuấn

Đối Tượng (tiếp theo)

- Thuộc tính
  - Tính chất mô tả một đối tượng
- Hành động
  - Dịch vụ mà đối tượng có thể đáp ứng
- Phương thức
  - Đặc tả cách đáp ứng bằng hành động khi được yêu cầu
- Thông điệp
  - Yêu cầu một hành động
- Biến cố
  - Sự kích thích từ đối tượng này gởi sang đối tượng khác

Đối tượng

- Khi một biến khai báo là một kiểu dữ liệu lớp → chưa sử dụng được → phải được cấp phát
- Cú pháp: **<tên biến> = new <tên lớp>(các tham số);**
- *new* là từ khóa khai báo;
- *<tên lớp>* là bắt buộc trùng với tên lớp
- còn gọi là phương thức khởi tạo;
- *(các tham số)* là tùy thuộc vào phương thức khởi tạo.

denm81@gmail.com

Đối tượng

SinhVien sv1;

sv1 = new SinhVien();

Phương thức khởi tạo mặc định

Từ khóa new

→đối tượng sv1 đã có trong bộ nhớ → kiểu tham chiếu

SinhVien sv1, sv2;

sv2 = sv1;

→thay đổi trong sv1 thì sv2 sẽ thay đổi theo

denm81@gmail.com

Object: Làm việc với class Instance

Book book = **new Book**("A Tale of Two Cities", 1859);

- toán tử **new** đã trả về một tham chiếu đến đối tượng vừa được tạo

Lớp Và Đối Tượng

- Lớp là một thực thể, còn đối tượng là một thực thể thực tế
- Lớp là một mô hình ý niệm định rõ các tính chất và các hành động được quy định bởi một đối tượng, còn đối tượng là một mô hình thực sự
- Lớp là khuôn mẫu từ đó đối tượng được tạo ra
- Tất cả các đối tượng trong cùng một lớp có các tính chất và các hành động như nhau

Hiệu chỉnh dữ liệu

- *class*: là từ khóa của java
- *ClassName*: là tên chúng ta đặt cho lớp
- *field_1, field_2*: các thuộc tính (các biến, hay các thành phần dữ liệu của lớp)
- *constructor*: là phương thức xây dựng, khởi tạo đối tượng của lớp.
- *method_1, method_2*: là các phương thức (có thể gọi là hàm) thể hiện các thao tác xử lý, tác động lên các thuộc tính của lớp.

```
class <ClassName>
{
    <kiểu dữ liệu> <field_1>;
    <kiểu dữ liệu> <field_2>;
    constructor
    method_1
    method_2
}
```


denm81@gmail.com

Lớp trong Java

- Cú pháp khai báo lớp (Class)

```
class Classname
{
        var_datatype variablename;
        :

        met_datatype
methodname(parameter_list)
        :
}
```

Lớp trong Java

```
//  comments about the class
public class MyProgram
{
}
```

class header

class body

Comments can be added almost anywhere

denm81@gmail.com

Lớp trong Java

- Cú pháp khai báo lớp (Class)

```
class tênlớp
{
        kiểu  tênbiến;
        :

        met_datatype      tênphươngthức      (ds
thamsố)
        :
}
```

Lớp: class

- từ khóa: *class*
- Cú pháp: **[phạm vi] [truy cập] class <tên lớp>{**

```
    //phần thân lớp, gồm: thuộc tính và hành vi
}
```

- **(1) Sự đóng gói (Encapsulation):** gom chung dữ liệu và chương trình vào trong một khối → đồng nhất giữa dữ liệu và các thao tác tác động lên dữ liệu đó.

denm81@gmail.com

Lớp: class

- Khai báo biến thuộc lớp: **<tên lớp> <tên biến>;**
- Thành phần của lớp:
  - Thuộc tính: là kiểu dữ liệu cơ sở hoặc là lớp khác;
  - Phương thức: là chức năng mà lớp có thể thực hiện.

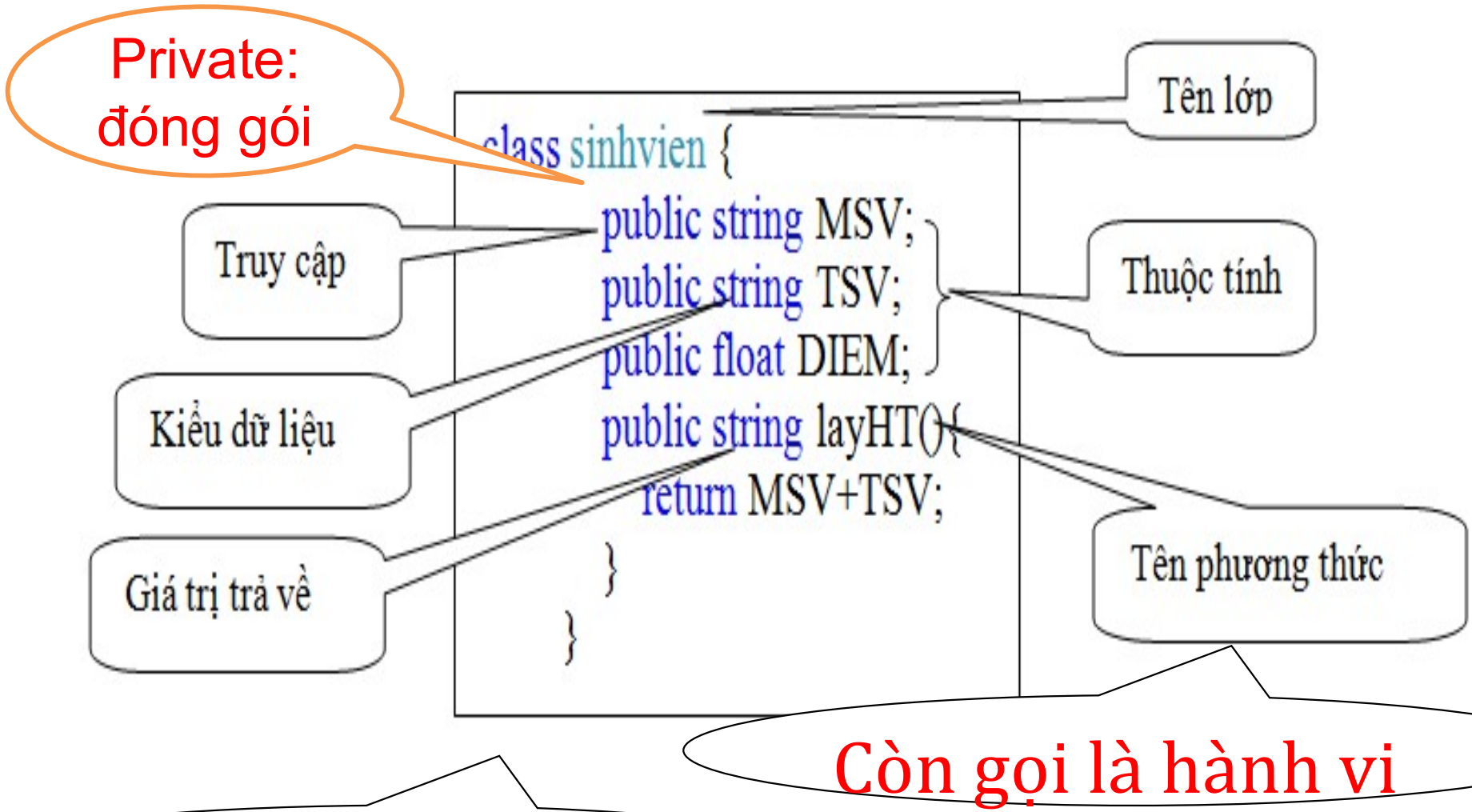
Ví dụ: xây dựng lớp sinhvien có 3 thuộc tính: mã sinh viên (chuỗi); tên sinh viên (chuỗi); điểm (số thực).

- Lớp sinh viên có phương thức để xếp loại sinh viên và trả về kết quả xếp loại.
- → khai báo: SV sinhvien1;

denm81@gmail.com

Lớp: class

Ví dụ: xây dựng lớp sinhvien có 3 thuộc tính



denm81@gmail.com

Constructor

- Tiến trình tạo ra một đối tượng được gọi là Constructor
- Một Constructor:
  - Cấp phát vùng nhớ
  - Khởi gán những thuộc tính (nếu có)
  - Cho phép truy cập những thuộc tính và phương thức

Destructor

- Tiến trình hủy một đối tượng gọi là Destructor
- Một Destructor:
  - Giải phóng bộ nhớ
  - Cấm truy cập thuộc tính và phương thức

Tính Lưu Trữ

- Tính lưu trữ là khả năng của đối tượng có thể lưu lại dữ liệu của nó sau khi đã bị hủy

Tính Bao Bọc Dữ Liệu

- Tiến trình che dấu những chi tiết hiện thực một đối tượng được gọi là tính bao bọc
- Ưu điểm:
  - Tất cả những thuộc tính và phương thức cần thiết đều được tạo
  - Một lớp có thể có nhiều tính chất và phương thức nhưng chỉ một số trong đó được hiển thị cho người dùng

denm81@gmail.com

Tính Kế Thừa

LỚP SINH VIÊN	LỚP NHÂN VIÊN	LỚP KHÁCH HÀNG
Tên	Tên	Tên
Địa chỉ	Địa chỉ	Địa chỉ
Điểm môn 1	Lương	Sản phẩm mua được
Điểm môn 2	Chức vụ	Nhập tên
Nhập tên	Nhập tên	Nhập địa chỉ
Nhập địa chỉ	Nhập địa chỉ	Nhập mã sản phẩm
Nhập điểm	Nhập lương	Lập hóa đơn
Tính tổng số điểm	Tính lương	

Tính Kế Thừa (tiếp theo)

Lớp Người
Tên
Địa chỉ
Nhập tên
Nhập địa chỉ

Tính Kế Thừa (tiếp theo)

LỚP NGƯỜI
Tên
Địa chỉ
Nhập tên
Nhập địa chỉ

+

Thêm các thuộc tính và hành động cần thiết vào lớp khách hàng
Nhập mã sản phẩm đã mua
Lập hóa đơn

=

Lớp Khách Hàng

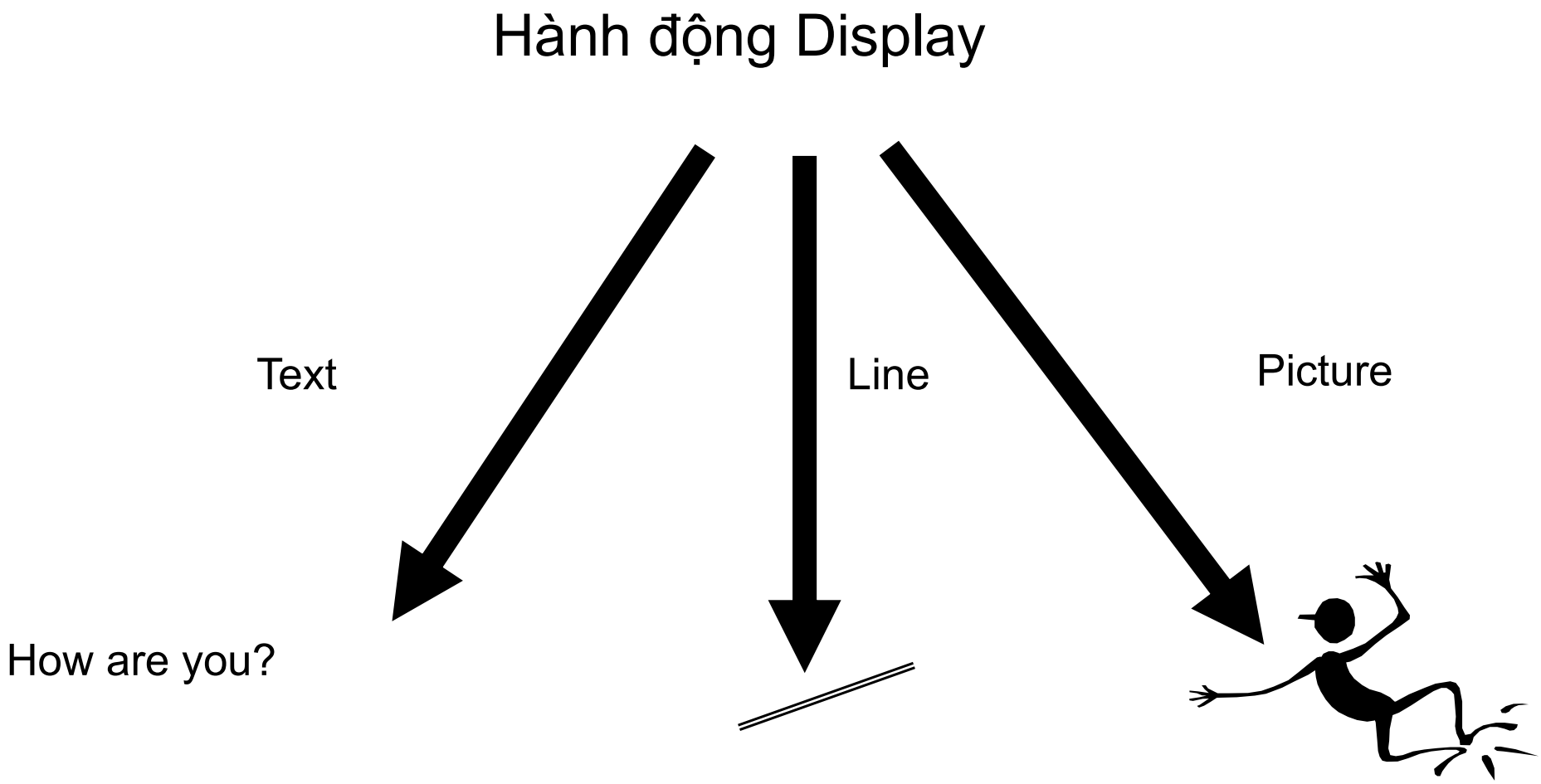
Tính Kế Thừa (tiếp theo)

- Tính Thừa kế
  - Là cơ chế cho phép một lớp chia sẻ những thuộc tính và những hành động đã định nghĩa trong một hoặc nhiều lớp khác
- Lớp con – dẫn xuất
  - Là lớp thừa kế từ lớp khác
- Lớp cha – gốc
  - Là lớp từ đó một lớp khác thừa kế các ứng xử của nó
- Đa thừa kế
  - Khi một lớp con thừa kế từ hai hoặc nhiều lớp (java không cho phép)

Khoa Công Nghệ Thông Tin

Tính Đa Hình

- Tính đa hình là thuộc tính cho phép một hành động ứng xử khác nhau trên các lớp khác nhau



Access Level

- Các cấp truy cập (access level).
  - **private**: Chỉ mã trong cùng một class với thành viên có thể truy cập thành viên.
  - **public**: Bất kỳ mã nào trong bất kỳ class nào trong bất kỳ package nào cũng có thể truy cập thành viên.
  - **protected**: Bất kỳ mã nào trong cùng một class hoặc các class con của nó đều có thể truy cập vào thành viên.

Getter Và Setter

- Hai tên gọi của hai thể loại phương thức
- Đặc tính ***gói ghém dữ liệu – Encapsulation***
- Giúp cho các thuộc tính *private*
  - cho đọc (thông qua phương thức *getter*)
  - cho ghi (thông qua phương thức *setter*)
- *Khả năng truy cập* cho các thuộc tính trong một lớp về ***private***.

Getter Và Setter

- *setter* → đặt tên phương thức là ***setTênThuộcTính()***.
- *getter* → đặt tên phương thức là ***getTênThuộcTính()***.

Getter Và Setter

```
public class ToaDo {
    private int x;
    private int y;
        public int getX() { // getter của thuộc tính x
        return x;
    }
       public void setX(int x) { // setter của thuộc tính x
        this.x = x;
    }
        public int getY() {// getter của thuộc tính y
        return y;
    }
        public void setY(int y) {// setter của thuộc tính y
        this.y = y;
    }
```

denm81@gmail.com

Getter Và Setter

```
class HinhTron {
    private final float PI = 3.14f;
    private float banKinh;
    public float getBanKinh() {
        return banKinh;
    }
    public void setBanKinh(float banKinh) {
        this.banKinh = banKinh;
    }
    public float tinhChuVi() {
        return 2 * PI * banKinh;
    }
    public float tinhDienTich() {
        return PI * banKinh * banKinh;
    }
}
```


denm81@gmail.com

Getter Và Setter

```
public class SinhVien {
    private String ten;
    private String tuoi;
    public String getTen() {
        return ten;
    }
    public void setTen(String ten) {
        if (ten == null || ten.isEmpty()) {
            this.ten = "Không biết";
        } else {
            this.ten = ten;
        }
    }
```

Getter Và Setter

```
    public String getTuoi() {
        if (Integer.valueOf(tuoi) != -1) {
            // Tuổi hợp lệ
            return tuoi;
        } else {
            return "Tuổi không hợp lệ";
        }
    }

    public void setTuoi(int tuoi) {
        // Kiểm tra tuổi có hợp lý hay không, nếu
        if (tuoi > 18) {
            this.tuoi = String.valueOf(tuoi);
        } else {
            this.tuoi = String.valueOf(-1);
        }
    }

}
```

Getter Và Setter

```
public class MainClass {

    public static void main(String[] args) {
        // Khởi tạo các đối tượng sinhVien
        SinhVien sinhVien1 = new SinhVien();
        SinhVien sinhVien2 = new SinhVien();

        // Set thông tin vào cho các sinh viên
        sinhVien1.setTen(""); // Tên không có
        sinhVien1.setTuoi(23);

        sinhVien2.setTen("Peter");
        sinhVien2.setTuoi(17);

        // In thông tin các sinh viên
        System.out.println("Sinh viên 1 có tên
        System.out.println("Sinh viên 2 có tên
    }
}
```


denm81@gmail.com

Phương thức nạp chồng: Method Overloading

- khai báo các phương thức có cùng tên nhưng với các danh sách tham số khác nhau
- Hai **method** cùng tên **bị overload** khi **danh sách tham số** của chúng khác nhau về **số lượng** hoặc **thứ tự tham số**.
- Hai phương thức cùng tên cũng bị overload khi có ít nhất một tham số khác nhau về kiểu.

Phương thức nạp chồng: Method Overloading

```
void draw(Shape shape)
{
    // drawing code
}
```

```
void draw(Shape shape, double x, double y)
{
    // drawing code
}
```

```
void draw(String string)
{
    // drawing code
}
```

```
void draw(String string, double x, double y)
{
    // drawing code
}
```

Phương thức nạp chồng: Method Overloading

- draw('abc') → chọn phương thức thứ ba vì nó cung cấp một danh sách tham số phù hợp.
- draw(null, 10, 20)→ báo cáo một thông báo lỗi 'reference to draw is ambiguous' do có hai method để chọn.
- Không thể overload một method bằng cách chỉ thay đổi kiểu trả về. Không thể chỉ định:

→ int add(int x, int y)

→double add(int x, int y)

Phương thức khởi tạo

Lớp CongDan (mọi lớp) luôn có sẵn phương thức khởi tạo (tên lớp)

```
SinhVien sv1;        sv1 = new SinhVien();
CongDan cd1; cd1 = new CongDan();
```

Thêm phương thức khởi tạo (khác tham số)

```
public class CongDan {
    int SCMND;
    string DanToc;
    0 references
    public CongDan(int scmnd) {
        SCMND = scmnd;
    }
}
```

```
sv1 = new SinhVien()
```

▲ 2 of 2 ▼ SinhVien(**int scmnd**)

denm81@gmail.com

Constructor: Initializing Objects

- Khai báo constructor bao gồm:
  - Một tiêu đề (Bao gồm tên class (Constructor thì không có tên riêng), sau đó là danh sách tham số tùy chọn)
  - Một body được phân định bằng dấu ngoặc nhọn.

```
className ( [parameterList] )
{
    // Phần thân constructor
}
```

Constructor: Initializing Objects

- Khai báo constructor bao gồm:
  - Một tiêu đề (Bao gồm tên class (Constructor thì không có tên riêng), sau đó là danh sách tham số tùy chọn)
  - Một body được phân định bằng dấu ngoặc nhọn.

```
className ( [parameterList] )
{
    // Phần thân constructor
}
```

Constructor: Initializing Objects

- Hàm Constructor mặc định không đối số
- class không khai báo bất kỳ hàm constructor nào, trình biên dịch sẽ tạo một hàm **constructor mặc định không có đối số**.
- Constructor mặc định này không làm gì cả
- Ví dụ sau khai báo một hàm constuctor trong class Book.

Constructor: Initializing Objects

• Ví dụ sau khai báo một hàm constuctor trong class Book.

```
class Book{
// ...
  Book (String _title, int _pubYear)  {
    title = _title;
    pubYear = _pubYear;
    ++count;
   }
}
```

Constructor: Initializing Objects

```
class Book{
  Book (String _title, int _pubYear)  {
        this.title = title;
        this.pubYear = pubYear;
        ++count;
    }
  void setTitle(String title){
        this.title = title;
  }
 void setPubYear(int pubYear){
        this.pubYear = pubYear;
 }
}
```

- this đại diện cho đối tượng hiện tại (thực sự, tham chiếu của nó).
- truy cập tên trường thay vì tham số cùng tên

Constructor: Initializing Objects

```
class Book{
  Book (String title)  {
        this(title, -1);
   }
  void setTitle (Str     title, int pubYear){
          setTitle(title);
          setPubYear(pubYear);
          ++count;
  }
}
```

- **sử dụng từ khóa this** theo sau là danh sách đối số được đặt dấu ngoặc để gọi hàm constructor thứ hai.

denm81@gmail.com

Từ khóa this

- *this* được dùng để tham chiếu đến đối tượng hiện hành của lớp
- Sử dụng theo 3 cách sau:
  a) *Biến thuộc tính bị che bởi tham số truyền*
  b) *Truyền đối tượng hiện hành vào một tham số của 1 phương thức của đối tượng*

denm81@gmail.com

Từ khóa this

a) *Biến thuộc tính* bị che bởi tham số truyền

```
class CongDan{
    int SCMND;
    String DanToc;
    public CongDan(int SCMND) {
        SCMND = this.SCMND;
    }
    public CongDan() {
    }
    public void DiHoc() {}
}
```


denm81@gmail.com

Từ khóa this

b) Truyền đối tượng hiện hành vào một tham số của 1 phương thức của đối tượng

```
class SinhVien extends CongDan{
    int SCMND;
    String DanToc;
    public void truyenDi(danhsachSVgioi ds) {
        ds.themSV(this);
    }
}
class danhsachSVgioi{
    public SinhVien[] ds;  int viTri; //
    public danhsachSVgioi(int sl){
        viTri = 0; //
        ds = new SinhVien[100];
    }
    public void themSV(SinhVien sv){//
      ds[viTri] = sv; //
       viTri++; //
     }
}
```


denm81@gmail.com

finalize

- Phương thức sẽ gọi ngay trước khi đối tượng bị hủy bởi Garbage Collector

```
@Override
protected void finalize() throws Throwable {
    // TODO Auto-generated method stub
    super.finalize();
}
```


denm81@gmail.com

Procedure-Oriented Programming

- Phương pháp lập trình theo kiểu cấu trúc = Phương pháp lập trình hướng thủ tục (Procedure-Oriented Programming)
- LTV phân tích một nhiệm vụ lớn thành nhiều công việc nhỏ hơn
- → chi tiết, cụ thể hoá để được các vấn đề đơn giản
- tìm ra cách giải quyết vấn đề dưới dạng những thuật giải → các chương trình con
- → nguyên lý lập trình từ trên xuống (*top-down)*
- thể hiện quá trình suy diễn từ cái chung cho đến cái cụ thể.

denm81@gmail.com

Object Oriented Programming – OOP

- Lập trình hướng đối tượng:
  - Dữ liệu và thao tác liên kết với nhau
  - công việc mà máy tính thực hiện → xử lý dữ liệu
  - mỗi thứ đều đòi hỏi ở thứ kia có mục tiêu cụ thể
  - hướng đối tượng làm tường minh mối quan hệ trên.
  - Lấy đối tượng làm nền tảng để xây dựng (thuật giải, chương trình)
  - suy nghĩ "*làm thế nào để giải quyết công việc*" ?
  - tư duy "*với một số đối tượng đã có, phải làm gì để giải quyết được công việc đặt ra*" → *làm cái gì với một số đối tượng đã có đó*

denm81@gmail.com

- Lập trình hướng đối tượng:

  – một chiếc xe có các bánh xe, di chuyển được và có thể đổi hướng của nó bằng cách quẹo tay lái.

  – một cái cây là một loại thực vật có thân gỗ và lá.

  – Một chiếc xe không phải là một cái cây, mà cái cây không phải là một chiếc xe

  – làm được với một chiếc xe thì không thể làm được với một cái cây.

denm81@gmail.com

- Lập trình hướng đối tượng:

  – Một mẫu tin (record) SV → đọc ra, thay đổi và lưu trữ lại theo yêu cầu ?

  – còn số phức thì có thể được dùng trong các tính toán.

  – viết một số phức vào tập tin làm mẫu tin nhân sự

  – và ngược lại hai mẫu tin nhân sự lại không thể cộng với nhau được.

  – Một chương trình hướng đối tượng sẽ xác định đặc điểm và hành vi cụ thể của các kiểu dữ liệu

denm81@gmail.com

Bài tập tại lớp

Yêu cầu: viết lớp CongDan (số chứng minh thư SCMT, Họ tên HoTen)

- Có Phương thức:
  - DiHoc (trả về "Học tại trường: ")
  - 4 phương thức khởi tạo là tổ hợp của (SCMT, HoTen)
  - Thuộc tính chỉ được lấy ra, nếu không có thì hiển thị 0, "chưa có"

denm81@gmail.com

Bài tập tại lớp

Yêu cầu: viết lại lớp CongDan có them yêu cầu

- Quản lý được số lượng đối tượng được tạo ra
- Các thuộc tính được xử lý (lấy ra/gán vào)

denm81@gmail.com

Bài tập tại lớp

Yêu cầu: viết lớp SinhVien (Mã sinh viên MSV, Điểm toán DiemToan) kế thừa CongDan

- Có Phương thức:

  – DiHoc (viết lại và có gọi phương thức cha "Học tại trường: Đại học")

  – phương thức khởi tạo (MSV) có gọi lại phương thức khởi tạo cha (SCMT, HoTen)

  – DiemToan: nhập vào trong miền [0, 10]; lấy ra bình thường

denm81@gmail.com

Bài tập về nhà

Yêu cầu: chương trình có

- CongDan (số chứng minh thư SCMT, Họ tên HoTen, Năm sinh NamSinh)
- ThiSinh (mã thí sinh MTS, Ưu tiên UT, Điểm trung bình DTB)
- Chức năng
  - Thêm CongDan, ThiSinh vào mảng riêng biệt
  - Xóa CongDan theo SCMT, ThiSinh theo MTS
  - Tìm theo các tổ hợp (SCMT, MTS, HoTen)

  → lập trình với C# theo các yêu cầu thiết kế lớp

denm81@gmail.com

Bài tập về nhà

Yêu cầu: viết lớp theo yêu cầu

- CongDan (số chứng minh thư SCMT, Họ tên HoTen, Năm sinh NamSinh)
  - các phương thức khởi tạo là tổ hợp của SCMT phải có và tổ hợp của (HoTen, NamSinh)
  - Thuộc tính chỉ được lấy ra, nếu không có thì hiển thị "chưa có" , 0

denm81@gmail.com

Bài tập về nhà

ThiSinh kế thừa từ CongDan

- ThiSinh (mã thí sinh MTS, Ưu tiên UT, Điểm trung bình DTB)
  - phương thức khởi tạo (MSV, UT, DTB) có gọi lại phương thức khởi tạo cha (SCMT, NamSinh)
  - DTB: nhập vào trong miền [0, 10]; lấy ra bình thường
  - Thuộc tính chỉ được lấy ra, nếu không có thì hiển thị "chưa có" , 0

Bài tập về nhà

– phương thức Nhập thí sinh mới (có đầy đủ thông tin) từ bàn phím vào

– phương thức tính điểm tổng DT: UT = 1 thì cộng 2, UT = 2 thì cộng 1, ngược lại 0

» DT = DTB + UT

– phương thức NhapHoc (tham số Điểm sàn): nếu thỏa 2 điều kiện (DT > Điểm sàn và >= 18 tuổi) thì "đậu", ngược lại "rớt"

– Phương thức Nhập thí sinh giỏi truyền chính đối tượng đi vào phương thức Thêm của lớp Thí sinh ưu tiên